HDOC - 71

Dynamic Matrix Operations Using Pointer Notation

Question / Problem Statement

Write a C program to dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation. Implement each operation as a separate function and release all allocated memory properly.

Algorithm / Logic

1. Input the number of rows and columns for Matrix A and Matrix B.
2. Dynamically allocate memory for both matrices using malloc().
3. Read matrix elements using pointer notation.
4. If both matrices have the same dimensions, call separate functions to perform addition and subtraction.
5. If the number of columns of Matrix A equals the number of rows of Matrix B, call a separate function to perform multiplication.
6. Each operation dynamically allocates its result matrix, calculates values using pointer notation, displays the result, and frees the result memory.
7. Free every row of Matrix A and Matrix B, then free their row-pointer arrays before terminating the program.

Functions Used

allocateMatrix() - dynamically allocates memory for a matrix.

readMatrix() - accepts matrix elements using pointer notation.

displayMatrix() - displays matrix elements.

addMatrices() - adds two matrices of equal order.

subtractMatrices() - subtracts two matrices of equal order.

multiplyMatrices() - multiplies two compatible matrices.

freeMatrix() - releases dynamically allocated matrix memory.

Header Files Used

#include <stdio.h> - standard input/output functions

#include <stdlib.h> - dynamic memory allocation and exit status

Complete C Program

```
#include <stdio.h>
#include <stdlib.h>

int **allocateMatrix(int rows, int cols) {
    int **matrix = (int **)malloc(rows * sizeof(int *));
```

```
    if (matrix == NULL) return NULL;

    for (int i = 0; i < rows; i++) {
        *(matrix + i) = (int *)malloc(cols * sizeof(int));
        if (*(matrix + i) == NULL) {
            for (int j = 0; j < i; j++)
                free(*(matrix + j));
            free(matrix);
            return NULL;
        }
    }
    return matrix;
}

void readMatrix(int **matrix, int rows, int cols, char name) {
    printf("Enter elements of Matrix %c:\n", name);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            scanf("%d", (*(matrix + i) + j));
}

void displayMatrix(int **matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++)
            printf("%d\t", *(*(matrix + i) + j));
        printf("\n");
    }
}

void freeMatrix(int **matrix, int rows) {
    for (int i = 0; i < rows; i++)
        free(*(matrix + i));
    free(matrix);
}

void addMatrices(int **a, int **b, int rows, int cols) {
    int **sum = allocateMatrix(rows, cols);
    if (sum == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(*(sum + i) + j) = *(*(a + i) + j) + *(*(b + i) + j);

    printf("\nAddition of matrices:\n");
    displayMatrix(sum, rows, cols);
    freeMatrix(sum, rows);
}

void subtractMatrices(int **a, int **b, int rows, int cols) {
    int **difference = allocateMatrix(rows, cols);
    if (difference == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(*(difference + i) + j) = *(*(a + i) + j) - *(*(b + i) + j);

    printf("\nSubtraction of matrices (A - B):\n");
    displayMatrix(difference, rows, cols);
    freeMatrix(difference, rows);
}

void multiplyMatrices(int **a, int **b, int r1, int c1, int c2) {
```

```
    int **product = allocateMatrix(r1, c2);
    if (product == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            *(*(product + i) + j) = 0;
            for (int k = 0; k < c1; k++)
                *(*(product + i) + j) +=
                    *(*(a + i) + k) * *(*(b + k) + j);
        }
    }

    printf("\nMultiplication of matrices:\n");
    displayMatrix(product, r1, c2);
    freeMatrix(product, r1);
}

int main(void) {
    int r1, c1, r2, c2;
    int **a, **b;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);

    if (r1 <= 0 || c1 <= 0 || r2 <= 0 || c2 <= 0) {
        printf("Matrix dimensions must be positive.\n");
        return 1;
    }

    a = allocateMatrix(r1, c1);
    b = allocateMatrix(r2, c2);

    if (a == NULL || b == NULL) {
        printf("Memory allocation failed.\n");
        if (a != NULL) freeMatrix(a, r1);
        if (b != NULL) freeMatrix(b, r2);
        return 1;
    }

    readMatrix(a, r1, c1, 'A');
    readMatrix(b, r2, c2, 'B');

    if (r1 == r2 && c1 == c2) {
        addMatrices(a, b, r1, c1);
        subtractMatrices(a, b, r1, c1);
    } else {
        printf("\nAddition and subtraction are not possible because dimensions differ.\n");
    }

    if (c1 == r2)
        multiplyMatrices(a, b, r1, c1, c2);
    else
        printf("\nMultiplication is not possible because columns of A are not equal to rows of B.
\n");

    freeMatrix(a, r1);
    freeMatrix(b, r2);
    return 0;
}
```

Explanation

The program represents each matrix as a pointer to pointers. Memory for the row pointers is allocated first, followed by separate dynamic allocation for every row. Matrix elements are accessed with expressions such as *(*(matrix + i) + j), which is pointer notation equivalent to matrix[i][j].

Addition and subtraction require matrices of identical dimensions. Multiplication requires the number of columns in Matrix A to equal the number of rows in Matrix B. Each operation is implemented in a separate function. The result matrix created by an operation is freed inside that function, while the two input matrices are released in main() using freeMatrix(). This prevents memory leaks.

Test Cases

Test Case	Matrix A	Matrix B	Expected Result / Condition
1	2x2: [1 2; 3 4]	2x2: [5 6; 7 8]	Add = [6 8; 10 12], Sub = [-4 -4; -4 -4], Mul = [19 22; 43 50]
2	2x3	3x2	Addition/subtraction not possible; multiplication possible.
3	2x3	2x2	Addition/subtraction not possible; multiplication not possible.

Sample Output

```
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8

Addition of matrices:
6       8
10      12

Subtraction of matrices (A - B):
-4      -4
-4      -4

Multiplication of matrices:
19      22
43      50
```

Result

Thus, a C program was successfully implemented to dynamically allocate two matrices and perform matrix addition, subtraction, and multiplication using pointer notation and separate functions. All dynamically allocated memory is released properly after use.